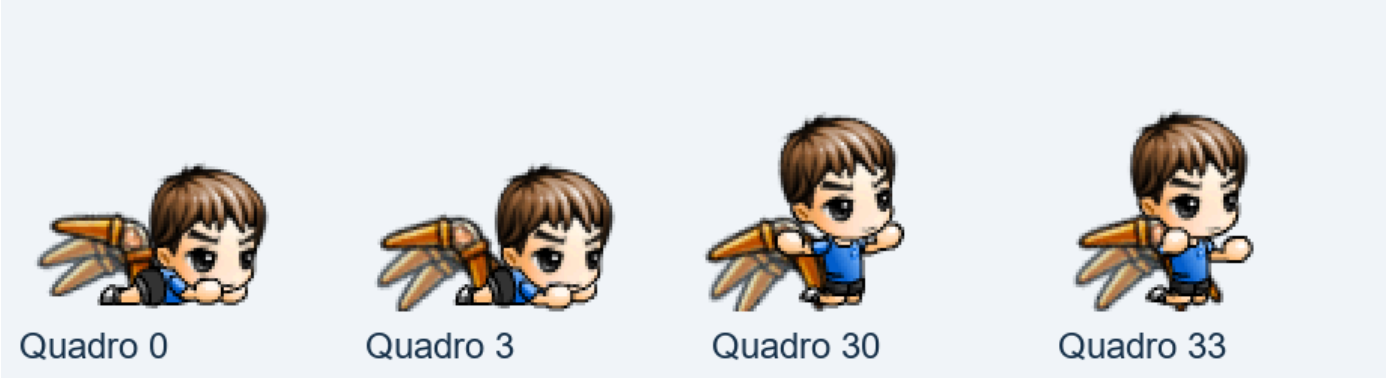
01 / CONCLUSÃO

Como o corpo funciona

Relatório ilustrado - montagem, caminhada e arremesso em combate

O mecanismo central é um compositor de sprites por quadros. A roupa já contém o corpo desenhado deitado, caminhando e arremessando. O programa recorta a pose adequada e sobrepõe as peças no mesmo espaço de coordenadas.



Recomposição dos PNGs arquivados: repouso (0), caminhada (3), preparação (30) e arremesso (33). Imagens ampliadas sem suavização para tornar os pixels e encaixes visíveis.

O que isso esclarece

O corpo não ganha a pose de combate apenas ao girar uma imagem em pé. As diferenças de pernas, braços, tronco e roupa estão pintadas nos quadros. O resultado depende da compatibilidade entre os atlas de todas as peças.



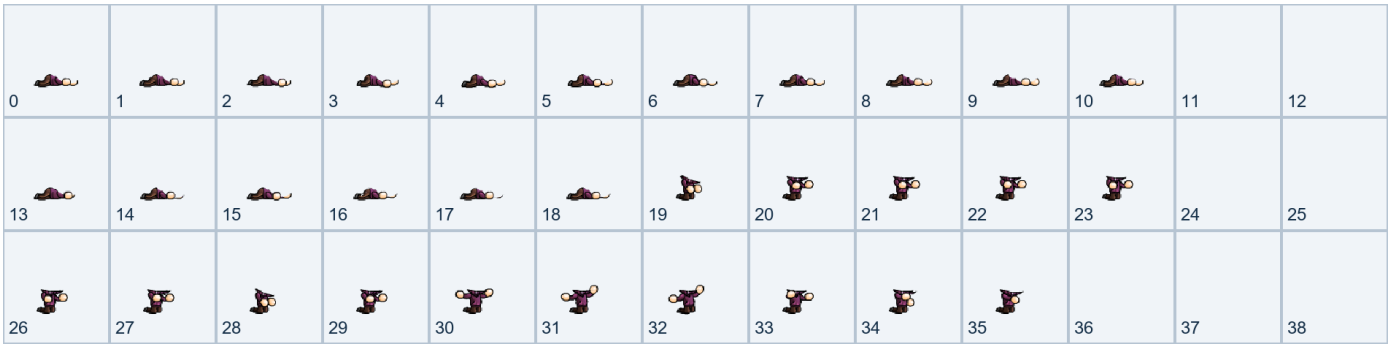
A mesma roupa 5102 tem arte de apresentação (SHOW) e arte de combate (GAME). São representações distintas do mesmo item; GAME contém várias poses, não apenas uma imagem deitada.

Evidência: GameCharacter.as, initSizeAndPics(), update() e drawFrame(); GameCharacterLoader.as, drawNormal(). Os recortes são dos arquivos disponíveis, não capturas de uma partida original.

39 posições por peça

Cada célula do combate mede 114 × 95 pixels

O código define 3 linhas e 13 colunas. O atlas completo tem 1482 × 285 px, medida também encontrada nos PNGs GAME usados aqui. Cada roupa guarda suas próprias versões dessas poses.



Atlas da roupa 5102, com índices adicionados para leitura. A grade e os números são explicativos; não fazem parte da textura original. Células vazias ou pouco diferentes não representam, por si só, ações ausentes.

Como encontrar um quadro

coluna = índice % 13
linha = floor(índice / 13)
Recorte: x = coluna × 114; y = linha × 95; largura = 114; altura = 95.

Exemplo: quadro 33 = coluna 7, linha 2, origem (798, 190). A mesma janela de recorte é aplicada aos atlas correspondentes.

Por que o encaixe se mantém

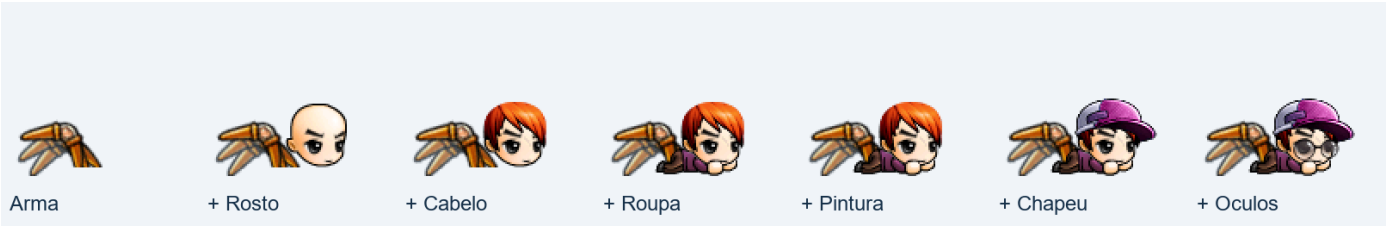
As peças foram desenhadas dentro de células compatíveis. O alinhamento não precisa ser reinventado por roupa em cada animação. Um port fiel preserva o tamanho da célula, o registro e a transparência, sem redimensionar cada peça isoladamente.

Evidência: GameCharacter.as:271-275; BaseCharacter.as:277-288; PNGs GAME da amostra. Os índices deste relatório começam em zero, como no código.

As camadas do personagem

Três grupos pré-compostos, depois o recorte de cada quadro

O carregador separa a imagem em grupos: faceDown = arma; face = rosto; faceUp = cabelo + roupa + pintura facial + chapéu + óculos. drawFrame() copia esses grupos para o bitmap final, respeitando a transparência.



Montagem cumulativa no quadro 3. Cada coluna acrescenta a peça seguinte. Cabelo 3102, roupa 5102, chapéu 1102 e óculos 2102; demais peças padrão. Sem tintura adicional.

Ordem normal, do fundo para a frente

Arma → rosto → cabelo → roupa → pintura facial → chapéu → óculos.
O termo “effect” neste catálogo é a pintura facial, não o impacto do projétil. As asas são tratadas separadamente como MovieClip.

Nem sempre todos usam o mesmo índice

Na caminhada e nas ações comuns, os três grupos usam o índice da sequência atual. No repouso STAND, arma e rosto usam STAND.frames[0], enquanto o grupo superior usa STAND.frames[1]. Portanto, sincronizar tudo em um único índice para todas as ações perde parte da lógica original.

Chapéus, cores e fantasia

O cabelo escolhe variante A ou B conforme getHairType()/Property1 do chapéu. As máscaras previstas em Property8 recebem tintura com ColorTransform e realce HARDLIGHT. A fantasia completa usa outro caminho de montagem e substitui as peças cosméticas normais; a arma continua incluída separadamente.

Evidência: GameCharacterLoader.as:65-89 e 138-244; GameCharacter.as:293-325 e 1000-1071; BaseLayer.as:60-150; PlayerInfo.as, getHairType().

Andar já é uma animação deitada

A pose visual e a posição no terreno são responsabilidades diferentes

A sequência WALK é [1, 1, 2, 2, 3, 3, 4, 4, 5, 5]. São cinco desenhos, cada um mantido por dois avanços da sequência. WALK_LACK_HP usa essa mesma lista no trecho analisado.



Personagem recomposto com as peças padrão e o bumerangue 7035. Observe que a roupa e o rosto mudam de posição dentro da célula ao longo dos quadros.

Um ciclo visual não desloca o jogador sozinho

PlayerWalkAction pede o próximo ponto válido com getNextWalkPoint(direction), atualiza info.pos e solicita a ação WALK. Ao chegar ao destino, encerra o movimento e volta a STAND. Isso separa a animação do corpo da navegação sobre o chão.

Fluxo que o HTML precisa reproduzir

Entrada / posição autorizada → próximo ponto do terreno → posição e direção → ação WALK → índice do quadro → composição das peças.

Espelhar o conjunto muda a direção visual. Calcular chão, colisão, inclinação e sincronização pertence ao objeto de jogo e às ações de movimento, não ao simples recorte do atlas.

Tempo: o que está confirmado

A lista confirma a duração relativa dos desenhos em passos de animação. Este relatório não fixa a duração em milissegundos: a taxa global do cliente original não foi comprovada nesta análise. O loop também passa por um ramo de reinício do índice, que deve ser considerado no port.

Evidência: GameCharacter.as, WALK/WALK_LACK_HP e update(); game/actions/PlayerWalkAction.as, execute() e finish(). Sem alegar port completo da física ou da rede.

Preparar, segurar e arremessar

As ações escolhem sequências distintas; não são uma rotação genérica

SHOWTHROWS: [28, 29, 30, 30, 30].
Preparação visual. STOPTHROWS: [34]. THROWS: [31, 32, 33, 34, 35]. HIDETHROWS: [35]. Os nomes e índices são os declarados no AS3; a ordem de acionamento depende do controlador do ataque.



Preparação: quadros 28, 29 e 30. As repetições do quadro 30 não estão duplicadas na ilustração.



Arremesso: quadros 31 a 35. O desenho da roupa inclui a mudança dos braços e do tronco; rosto e demais peças acompanham a sequência.

Há também o caminho de disparo



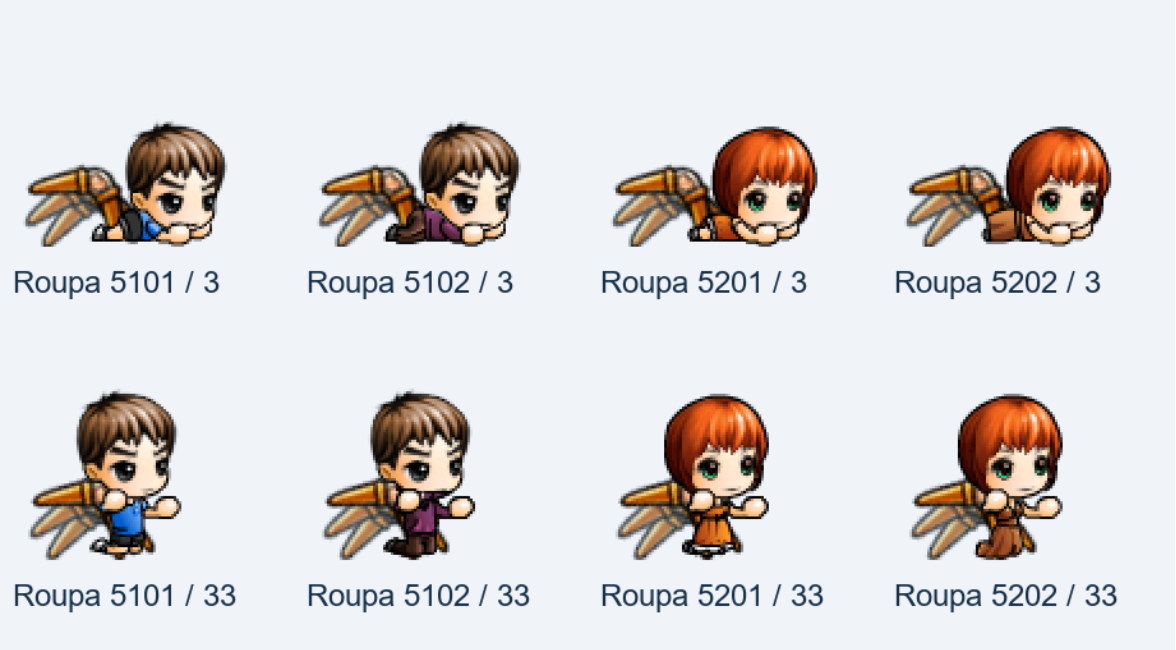
SHOWGUN: [19, 20, 21, 21, 21]; SHOT: [22, 23, 26, 27]; STOPSHOT: [23]; HIDEGUN: [27]. A faixa acima mostra os desenhos únicos envolvidos.

Controle de fim: PlayerAction guarda repeat, stopAtEnd e replaceSame. THROWS/SHOT não repetem e param no fim; a troca para outra ação é responsabilidade do controlador. O projétil, seu impacto e o dano são sistemas separados do bitmap do corpo.

Evidência: GameCharacter.as, declarações SHOWTHROWS/THROWS/SHOWGUN/SHOT; ddt/command/PlayerAction.as:13-33. Não foi inferido o instante exato de lançamento a partir da pose.

O que deve ser preservado

Conclusões para uma implementação fiel em HTML



Quatro roupas disponíveis na amostra (5101, 5102, 5201 e 5202), nos mesmos quadros 3 e 33. Cada uma tem seus próprios desenhos de combate; o contrato comum permite a troca de peças.

O que falta para chamar de réplica do compositor

O projeto atual usa recortes e transformações de partes em avatar.ts/appearance.ts. Isso é uma solução própria; não equivale ao compositor de 39 quadros do AS3. Para aproximar o comportamento original, é necessário organizar atlas compatíveis por peça, portar as tabelas de ações e manter os três grupos de composição, inclusive a exceção STAND.

Plano técnico: (1) contrato de célula e registro; (2) catálogo e variantes; (3) pré-composição e máscaras; (4) sequências e regras de transição; (5) posição/direção/chão separadas; (6) comparação quadro a quadro de várias roupas. Em Canvas 2D, drawImage com recorte substitui copyPixels. WebGL pode usar os mesmos retângulos de textura.

Limites e origem das imagens

A lógica vem do AS3 local em DDTank4.1-main/Source Flash/scripts. As figuras são recomposições dos PNGs GAME arquivados na pesquisa anterior, com proveniência no manifest.json. O catálogo ilustrado é uma amostra, não todas as roupas do DDTank. SHOW tem divergência de altura entre o código e os PNGs disponíveis; não se presume uma única revisão para tudo.

Este relatório separa o conhecimento adquirido. Não altera o jogo nem integra esses assets ao catálogo. A técnica pode ser reproduzida com a arte original do projeto. A lista de arquivos e hashes usados acompanha o PDF em ddtank-rig-imagens/fontes.json.

Referências principais: ddt/view/character/{GameCharacter,GameCharacterLoader,BaseCharacter,BaseLayer}.as; ddt/command/PlayerAction.as; game/actions/PlayerWalkAction.as.